Ex. No. 10	DEVELOPING SERVER-SIDE APPLICATION USING NODE.JS
Date of Exercise	13/10/2026
GitHub Host Link	https://github.com/adhishm-0/web-technology-lab/tree/main/EXP10

Objective

To develop a basic server-side application using Node.js and implement HTTP request handling to deliver responses to clients.

Description

Node.js is a JavaScript runtime environment that allows JavaScript code to execute outside a web browser. The built-in http module can create an HTTP server, process client requests, and send responses without an additional framework. This experiment builds a small StarVerse server that returns HTML, displays request details, and serves different content at /home and /about.

KEY CONCEPTS USED :

• http.createServer() — create a server and handle each incoming request

• request (req) — read the requested URL and HTTP method

• response (res) — set response headers and send HTML to the client

• res.writeHead() — set the HTTP status and response headers

• res.end() — send the response body and finish the response

• server.listen() — start listening on port 3000

• URL.pathname — choose a response for /, /home, or /about

Procedure

1. Create a folder named EXP10_Node_Server_Application.

2. Create server.js and import the built-in http module.

3. Define StarVerse page content for /, /home, and /about.

4. Create the server callback to select a route and send an HTML response.

5. Start the server on port 3000 using server.listen().

6. Run node server.js in Konsole.

7. Open http://localhost:3000 and verify the root, /home, and /about responses.

Run command

node server.js

No additional packages are required; the application uses Node.js's built-in http module.

Program Code: server.js

The server uses StarVerse, matching the website name in Experiments 1–3.

```
const http = require('http');

const PORT = 3000;
const routes = {
  '/': { title: 'Welcome to StarVerse!', message: 'This is a server-side application.' },
  '/home': { title: 'Welcome home to StarVerse!', message: 'Explore your astrology and horoscope
  experience.' },
  '/about': { title: 'About StarVerse', message: 'StarVerse is an astrology and horoscope website.' }
};

function escapeHtml(value) {
  return String(value).replace(/[&<>"']/g, (character) => ({
    '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;'
  })[character]);
}

function createPage(req, route, content, status) {
  return `<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>${escapeHtml(content.title)} | StarVerse</title>
  <style>
    * { box-sizing: border-box; }
    body { margin: 0; min-height: 100vh; display: grid; place-items: center; padding: 28px; color: #172554;
  background: #f5f7fb; font: 16px/1.6 Arial, sans-serif; }
    main { width: min(700px, 100%); padding: 52px; border: 1px solid #e3e8f1; border-radius: 20px;
  background: #fff; box-shadow: 0 20px 60px #1e315012; }
    .brand { color: #314d91; font-size: 14px; font-weight: 700; letter-spacing: .14em; }
    .eyebrow { margin-top: 44px; color: #7b88a3; font-size: 11px; font-weight: 700; letter-spacing: .16em;
  text-transform: uppercase; }
    h1 { margin: 8px 0 12px; font-size: clamp(32px, 6vw, 48px); line-height: 1.1; }
    .message { margin: 0 0 28px; color: #61708d; font-size: 18px; }
    .request { padding-top: 18px; border-top: 1px solid #e8ebf2; color: #5c6880; font-size: 14px; }
    .request p { margin: 5px 0; } .request strong { color: #253967; }
    nav { display: flex; gap: 12px; margin-top: 24px; } nav a { color: #314d91; font-weight: 700;
  text-decoration: none; }
    nav a:hover { text-decoration: underline; }
    @media (max-width: 540px) { main { padding: 30px 24px; } .eyebrow { margin-top: 32px; } }
  </style>
</head>
<body>
  <main>
    <div class="brand">✦ STARVERSE</div>
    <div class="eyebrow">Node.js HTTP Server · ${escapeHtml(route)}</div>
    <h1>${escapeHtml(content.title)}</h1>
    <p class="message">${escapeHtml(content.message)}</p>
    <div class="request">
      <p><strong>Request URL:</strong> ${escapeHtml(req.url)}</p>
      <p><strong>Request Method:</strong> ${escapeHtml(req.method)}</p>
      <p><strong>Response Status:</strong> ${status} ${status === 200 ? 'OK' : 'Not Found'}</p>
    </div>
    <nav><a href="/home">Home</a><a href="/about">About</a></nav>
  </main>
</body>
</html>`;
}

const server = http.createServer((req, res) => {
  const path = new URL(req.url, `http://${req.headers.host || 'localhost'}`).pathname;
  const page = routes[path];
  const status = page ? 200 : 404;
  const content = page || { title: 'Page not found', message: 'The requested StarVerse page does not
  exist.' };

  res.writeHead(status, { 'Content-Type': 'text/html; charset=utf-8' });
  res.end(createPage(req, path, content, status));
});

server.listen(PORT, () => console.log(`Server running at http://localhost:${PORT}`));
```

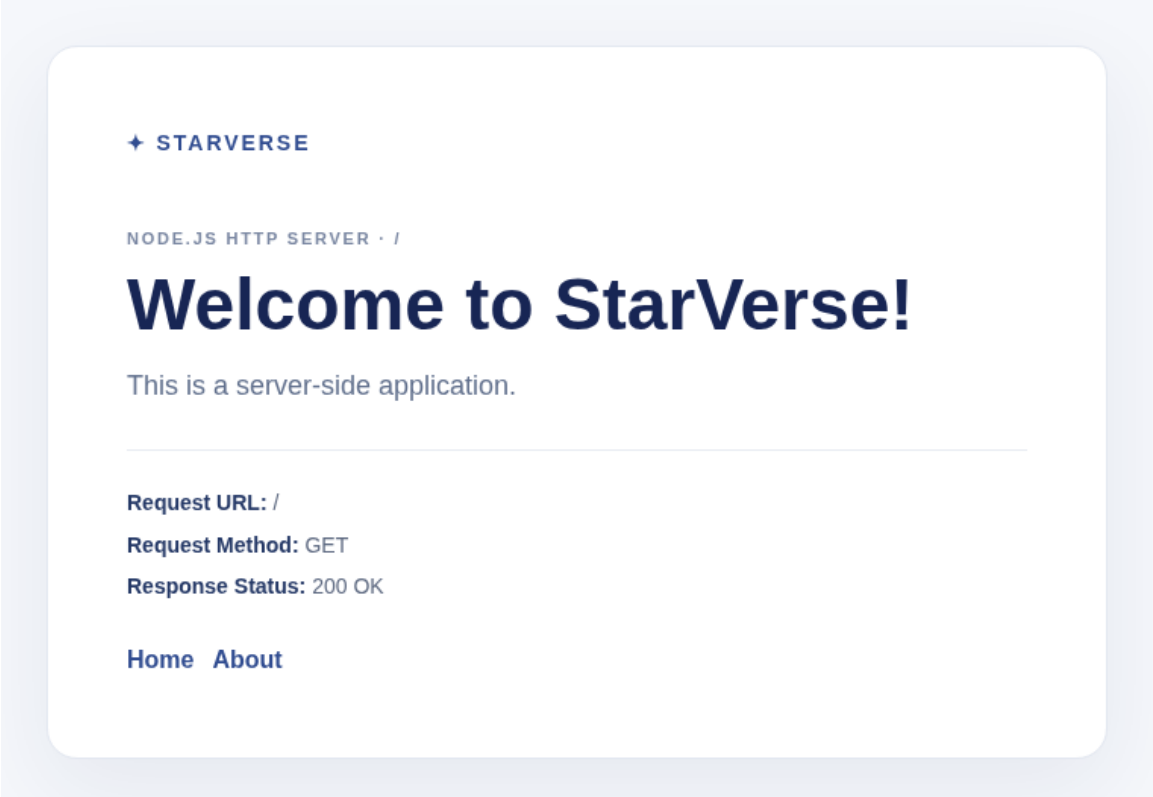
Program Output

Konsole — server startup

```
Server running at http://localhost:3000
```

Terminal output: Server running at http://localhost:3000

Browser Output — http://localhost:3000/



The page displays the StarVerse welcome message, request URL (/), request method (GET), and response status.

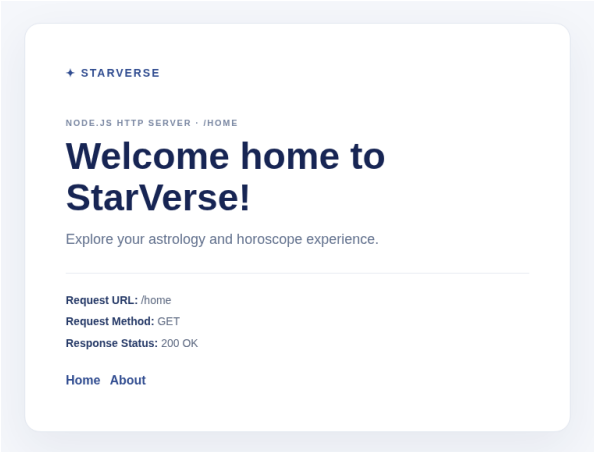
Question 5: Route-Based Responses

The route table returns a different StarVerse message for /home and /about. Unknown paths receive an HTTP 404 response.

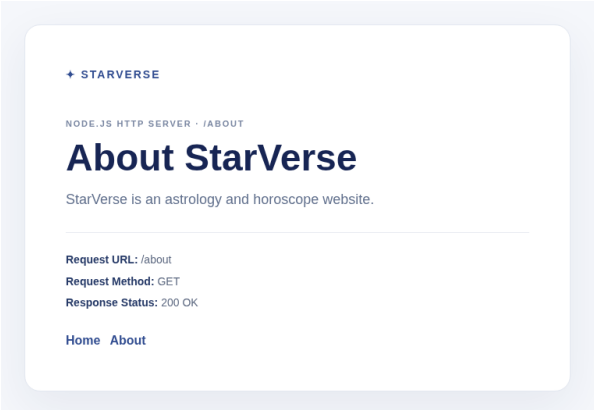
Program code

```
const routes = {
  '/home': {
    title: 'Welcome home to StarVerse!',
    message: 'Explore your astrology and horoscope experience.'
  },
  '/about': {
    title: 'About StarVerse',
    message: 'StarVerse is an astrology and horoscope website.'
  }
};
const requestUrl = new URL(req.url, `http://${req.headers.host || 'localhost'}`);
const page = routes[requestUrl.pathname];
const status = page ? 200 : 404;
res.writeHead(status, { 'Content-Type': 'text/html; charset=utf-8' });
res.end(createPage(req, requestUrl.pathname, page || notFoundPage, status));
```

Browser Output — /home and /about



http://localhost:3000/home



http://localhost:3000/about

Sample Questions and Answers

1. What is Node.js, and why is it used for server-side development?

Node.js is a JavaScript runtime built on the V8 engine. It runs JavaScript outside a browser and is used for servers and APIs because its event-driven, non-blocking I/O model can handle many connections efficiently.

2. What is the purpose of the built-in http module?

The http module provides the basic tools to create HTTP servers and clients. In this experiment it receives browser requests and sends status codes, headers, and HTML responses.

3. Explain http.createServer() and server.listen().

http.createServer(callback) creates a server and invokes the callback for each request, passing request and response objects. server.listen(port, callback) binds the server to the specified port and runs the callback after startup.

4. What is the purpose of res.writeHead() and res.end()?

res.writeHead(statusCode, headers) sets the response status and headers such as Content-Type. res.end(body) sends the response body and signals that the response is complete.

5. Modify the program to display different messages for /home and /about.

The route table in the program maps /home and /about to different StarVerse titles and messages. The screenshots on the previous page show both responses.

Result

A basic server-side application was developed and executed successfully using Node.js. The StarVerse HTTP server handles client requests, serves the root, /home, and /about routes, and returns HTML responses that include request details.